THE CLAUDE TEAM · @SHAZIMBUILDS

328 skills installed. I used 18.

I went through 60 days of my own Claude Code logs. Most of what I'd installed never got opened once. The work came from a short list of tools: browsers, the terminal, a video editor and a few connectors. Here are the 25 that did the work, organised like a team, with the install step and official link for each. At the end is the script I used, so you can run the same audit on your setup.

45	18 / 328	6,844
sessions in 60 days	installed skills used	terminal commands

How to read the numbers

Every count is a real tool call from my logs, 28 Aug to 27 Oct 2026, on one Mac. A skill counts once each time Claude loads it, so a skill with 4 calls was used in 4 separate jobs. A browser or connector counts every single action, which is why those numbers run into the thousands. Big numbers mean busy, not better.

THE CEO

Claude Code

Claude Code CEO

Everything below reports to it. I run it in the Claude desktop app's Code tab. It's also a terminal command.

INSTALL

The desktop app from `claude.com/download`, or in a terminal: `curl -fsSL https://claude.ai/install.sh | bash`

DEPARTMENT 1 OF 5 · BUILD

Build & Ship

Write it, test it, put it live.

01 Git Version Control

466 CALLS

I commit every working step, so any bad change is one command away from undone.

INSTALL

Mac: `xcode-select --install` (Apple's command line tools include Git), or `brew install git`

git-scm.com

02 GitHub CLI Release Manager

89 CALLS

Claude opens the pull request, writes the description and checks CI without me touching the browser.

INSTALL

`brew install gh` then `gh auth login`

cli.github.com

03 Docker Ops Lead

61 CALLS

Runs the database and services for a client build locally, the same way they run in production.

INSTALL

Download Docker Desktop for Mac or Windows

docs.docker.com/desktop

04 **Playwright** Test Pilot — 57 CALLS

Opens the app in a real browser and checks it works. It also renders every carousel I post, including this one.

INSTALL

`npm i -D playwright` then `npx playwright install chromium`

playwright.dev

05 **Vercel** Launch Engineer — 35 CALLS

One command puts a site or preview live, so a client can click a link the same day.

INSTALL

`npm i -g vercel` then `vercel login`

vercel.com/docs/cli

Start with one: GitHub CLI. Once Claude can open the pull request itself, shipping stops needing you in the browser.

DEPARTMENT 2 OF 5 · RESEARCH

Research & Read

Eyes on the web before anything gets written.

06 **Claude in Chrome** Browser Operator — 1,818 CALLS

My most used hire. Claude works inside the Chrome I'm already logged into: dashboards, forms, admin panels.

INSTALL

Add the extension from the Chrome Web Store and sign in. In Claude Code, run `/chrome` (or start with `claude --chrome`). Needs a paid Claude plan.

code.claude.com/docs/en/chrome

07 Built-in Browser Researcher

775 CALLS

Claude's own separate browser in the desktop app. Research and testing happen there, away from my logged-in accounts.

INSTALL

Nothing to install. It ships with the Claude desktop app (Code tab).

claude.com/download

08 Firecrawl Web Reader

149 CALLS

Reads a web page and hands Claude clean text, so research doesn't drown in menus and cookie banners.

INSTALL

`claude mcp add firecrawl -e FIRECRAWL_API_KEY=your-key -- npx -y firecrawl-mcp`
(key from the Firecrawl dashboard)

docs.firecrawl.dev/mcp-server

09 gstack browse Fast Checker

90 CALLS

A fast headless browser for quick checks: does the page load, what does it say, any console errors.

INSTALL

`git clone --single-branch --depth 1 https://github.com/garrytan/gstack.git ~/.claude/skills/gstack && cd ~/.claude/skills/gstack && ./setup` (needs Bun)

github.com/garrytan/gstack

10 **Notion** Archivist

16 CALLS

Claude reads my content system and the daily AI news feed before it writes anything.

INSTALL

`claude mcp add --transport http notion https://mcp.notion.com/mcp` then run `/mcp` to sign in

developers.notion.com

Start with one: Claude in Chrome. It's the one that does the chores you do in a browser every day.

DEPARTMENT 3 OF 5 · DESIGN

Design & Polish

So nothing ships looking AI-made.

11 **ImageMagick** Image Fixer

155 CALLS

Resizes, crops and converts every image before it goes into a post or a site.

INSTALL

```
brew install imagemagick
```

imagemagick.org

12 **artifact-design** Page Designer

6 CALLS

Gives the pages I publish from Claude a proper layout instead of a default one.

INSTALL

Nothing to install. It comes with Artifacts in the Claude desktop app.

claude.com/download

13 **ui-ux-pro-max** Design Lead — 4 CALLS

Spacing, type and colour rules Claude follows when it builds an interface.

INSTALL

In Claude Code: `/plugin marketplace add nextlevelbuilder/ui-ux-pro-max-skill` then `/plugin install ui-ux-pro-max@ui-ux-pro-max-skill`

github.com/nextlevelbuilder

14 **impeccable** Style Guardian — 4 CALLS

Audits a finished design and fixes the details that make it look AI-made.

INSTALL

`npx impeccable install` in your project, then `/impeccable init`

impeccable.style

15 **taste-skill** Taste Maker — 2 CALLS

Blocks the generic look: default fonts, purple gradients, everything centred.

INSTALL

`npx skills add Leonxlnx/taste-skill`

github.com/Leonxlnx/taste-skill

Start with one: impeccable. Run it on anything you've already built and read what it flags.

DEPARTMENT 4 OF 5 · MAKE

Make the Content

Reels, motion and the posting queue.

16 **ChatCut** Video Editor — 1,182 CALLS

Edits my reels from the terminal: cuts, captions, B-roll, export. My second most used hire.

INSTALL

In Claude Code: `/plugin marketplace add ChatCut-Inc/agent-plugin` then `/plugin install chatcut@chatcut-inc`, and sign in when asked

chatcut.io/claude

17 **ffmpeg** Cutter — 274 CALLS

Cuts, joins, compresses and converts video and audio. It made the animated cover of this carousel.

INSTALL

`brew install ffmpeg`

ffmpeg.org

18 **Postiz** Scheduler — 50 CALLS

Claude queues approved posts to Instagram, TikTok and LinkedIn. I still read every post before it goes in.

INSTALL

`npm install -g postiz` then `postiz auth:login`. Agent skill: `npx skills add gitroomhq/postiz-agent`

docs.postiz.com

19 **yt-dlp** Footage Runner — 13 CALLS

Downloads reference clips so I can study how a video is paced. Only use it on content you have the right to download.

INSTALL

`brew install yt-dlp`

github.com/yt-dlp/yt-dlp

20 HyperFrames Motion Designer

9 CALLS

Turns an HTML page into a rendered video, for motion graphics I can edit as code.

INSTALL

`npx skills add heygen-com/hyperframes` (needs Node 22+ and ffmpeg)

hyperframes.heygen.com

Start with one: ffmpeg. It's free, and Claude already knows every flag.

DEPARTMENT 5 OF 5 · SELL

Find & Sell

Leads in, replies out.

21 La Growth Machine SDR

106 CALLS

Claude checks my outreach sequences, reads replies and tells me who needs an answer today.

INSTALL

Follow La Growth Machine's Claude Code guide (needs an LGM account)

lagrowthmachine.com

22 Instantly Email Ops

23 CALLS

Cold email campaigns, analytics and inbox warm-up, managed from the terminal.

INSTALL

Create a key in Instantly (Integrations, API Keys), then `claude mcp add --transport http instantly https://mcp.instantly.ai/mcp --header "Authorization: YOUR_API_KEY"`

help.instantly.ai

23 n8n Automation Builder

9 CALLS

Claude looks up nodes and checks workflows while I build automations for clients.

INSTALL

In n8n: Settings, Instance-level MCP, Connect a client. Then switch on MCP access for each workflow you want Claude to see.

docs.n8n.io

24 linkedin-skills Copy Editor

3 CALLS

The humanizer pass that strips AI tells from my LinkedIn posts before I publish.

INSTALL

In Claude Code: `/plugin marketplace add sergebulaev/linkedin-skills` then `/plugin install linkedin-skills@linkedin-skills`

github.com/sergebulaev/linkedin-skills

25 OpenReply Inbox Runner

MY PRODUCT

My own product. You commented TEAM, it sent you this guide. Comment-to-DM for Instagram.

INSTALL

Sign up at reply.orbiqon.com

reply.orbiqon.com

Start with one: La Growth Machine or Instantly, whichever you already pay for. Ask Claude who needs a reply today.

BONUS

Audit your own setup

Takes about ten seconds. Nothing leaves your computer: it only reads the logs Claude Code already keeps in `~/.claude/projects`.

1. Save the script below as `audit_claude.py` (or ask Claude Code to create it for you).
2. Run `python3 audit_claude.py` for the last 60 days, or `python3 audit_claude.py 30` for 30.
3. Read the last section, "installed but never used". Anything there you don't recognise, remove it. Every installed skill adds to what Claude reads before it decides what to do.

```
#!/usr/bin/env python3
"""Audit your Claude Code setup: what you installed vs what you actually used.

Reads the session logs Claude Code keeps in ~/.claude/projects (nothing leaves
your machine) and prints:
  1. skills you loaded, with counts
  2. skills you installed but never loaded
  3. MCP servers / connectors by number of tool calls
  4. the command-line tools Claude ran most

Usage:  python3 audit_claude.py            (last 60 days)
        python3 audit_claude.py 30         (last 30 days)
"""
import collections, datetime, glob, json, os, re, sys

DAYS = int(sys.argv[1]) if len(sys.argv) > 1 else 60
HOME = os.path.expanduser("~/.claude")
since = (datetime.datetime.now(datetime.timezone.utc) -
datetime.timedelta(days=DAYS)).strftime("%Y-%m-%d")

skills, mcp, cli = collections.Counter(), collections.Counter(), collections.Counter()
sessions, days = set(), set()
HEREDOC = re.compile(r"<<-?\s*['\"]?(\w+)['\"]?[^\n]*\n.*?\n\s*\1\b", re.S)
PROGRAM = re.compile(r"^[a-z][\w.+-]*$")
SHELL = set("for do done if then else elif fi while until case esac in echo printf cd export
local "
            "set unset true false sleep read source test return exit wait trap eval import from
def print".split())

for path in glob.glob(f"{HOME}/projects/*/*.jsonl"):
    for line in open(path, errors="ignore"):
        try:
            row = json.loads(line)
        except ValueError:
            continue
        if (row.get("timestamp") or "") < since:
            continue
        msg = row.get("message")
        blocks = msg.get("content") if isinstance(msg, dict) else None
        if not isinstance(blocks, list):
            continue
```

```
        for b in blocks:
            if not isinstance(b, dict) or b.get("type") != "tool_use":
                continue
            sessions.add(path)
            days.add(row["timestamp"][:10])
            name, args = b.get("name", ""), b.get("input") or {}
            if name == "Skill":
                skills[args.get("skill", "?")] += 1
            elif name.startswith("mcp__"):
                mcp[name.split("__")[1]] += 1
            elif name == "Bash":
                cmd = HEREDOC.sub("", args.get("command") or "")
                ran = set()
                for part in re.split(r"&&|\|\||;|\||\n|\$\(|`", cmd):
                    words = [w for w in part.split() if "=" not in w]
                    while words and words[0] in ("sudo", "npx", "time", "env", "exec"):
                        words = words[1:]
                    if words:
                        tool = os.path.basename(words[0])
                        if PROGRAM.match(tool) and tool not in SHELL:
                            ran.add(tool)
                for tool in ran:  # count each tool once per command
                    cli[tool] += 1

# installed skills: personal skills + skills inside installed plugins
installed = {os.path.basename(os.path.dirname(p)) for p in glob.glob(f"
{HOME}/skills/*/SKILL.md")}
try:
    plugins = json.load(open(f"{HOME}/plugins/installed_plugins.json"))
    for key, entries in plugins.get("plugins", plugins).items():
        for e in entries if isinstance(entries, list) else [entries]:
            for p in glob.glob(f"{e.get('installPath', '')}/**/SKILL.md", recursive=True):
                installed.add(key.split("@")[0] + ":" + os.path.basename(os.path.dirname(p)))
except (OSError, ValueError, AttributeError):
    pass

used = {s for s in skills if s in installed}
never = sorted(s for s in installed if s not in used)

print(f"\nLast {DAYS} days: {len(sessions)} sessions on {len(days)} days\n")
print(f"SKILLS: {len(installed)} installed, {len(used)} of them used at least once")
for s, n in skills.most_common():
    print(f"  {n:>5}  {s}{'' if s in installed else '   (built-in or project skill)'}")
print(f"\nMCP SERVERS / CONNECTORS (tool calls)")
for s, n in mcp.most_common(15):
    print(f"  {n:>5}  {s}")
print(f"\nCOMMAND-LINE TOOLS CLAUDE RAN MOST")
for s, n in cli.most_common(20):
    print(f"  {n:>5}  {s}")
print(f"\nINSTALLED BUT NEVER USED ({len(never)}):")
print("  " + ", ".join(never) if never else "  none")
print("\nTip: connectors added in claude.ai show up as long ids. Match them by the")
print("tool names in your logs, or check Settings > Connectors.\n")
```

Want a team like this working inside your business?
That's what we build at **Orbiqon**. Reply to the DM and tell me the job you'd hand over first.

@SHAZIMBUILDS